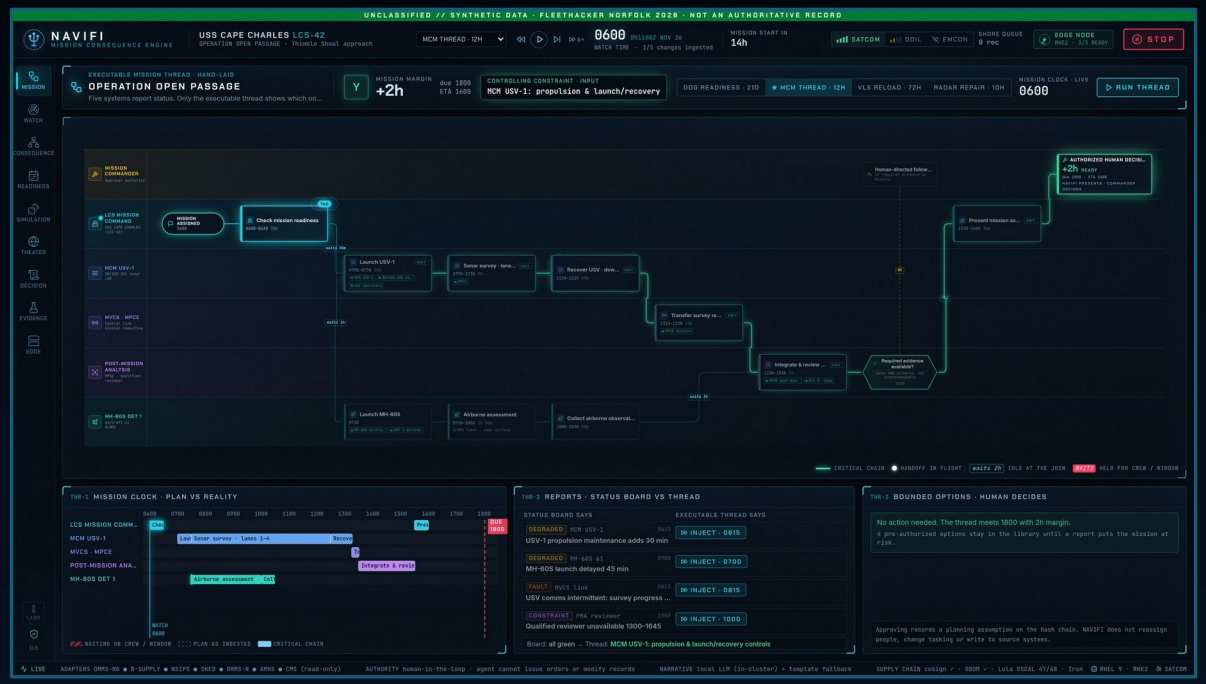
FLEETHACKER NORFOLK 2026

NAVIFI

A mission consequence engine for the watch floor

Know before it breaks the mission.

We built it this week: the app, the engine under it, and a real cluster to run it on. This deck is what we did, what we hit, and where it stands.



TUESDAY

Engine and watch floor

Consequence graph, Monte Carlo, hash-chained decision log, evidence pack, three scenarios

WEDNESDAY

The mission thread

Executable MCM thread, auto-laid threads, Connor's standalone mission engine, the video

THURSDAY

Live on our own cluster

RHEL 9, RKE2 and UDS Core stood up on AWS; NAVIFI deployed from a git commit

WHO WE ARE

A two-day sprint

Questions, or a mission you want to see run through it? Email either of us.



Bryce Schumacher

Built the NAVIFI app: the watch floor, the mission thread, the decision log and the evidence screens. Stood up the lab cluster on AWS.

bschumacher@uncomn.com



Connor Barnsley

Built the standalone mission engine: the dependency graph, Monte Carlo, the decision ledger, and naval and airlift mission packs, with 48 tests.

cbarnsley@uncomn.com

WHERE WE STARTED

Four reports land. Which one breaks the mission?

Today a Sailor answers that by hand. We wanted the watch to see it on one screen.

How it gets answered today

Pull the CASREP

↓

Call supply

↓

Check the watchbill

↓

Redo the whiteboard

30–60 min

per change, by our own tabletop estimate. Not measured fleet data.

What we built instead

NAVIFI reads the records of eight systems, read-only, and recomputes the mission in milliseconds.

Every number links back to the record it came from. Nothing is typed twice or written back.



OMMS-NG · R-SUPPLY · NSIPS · SKED

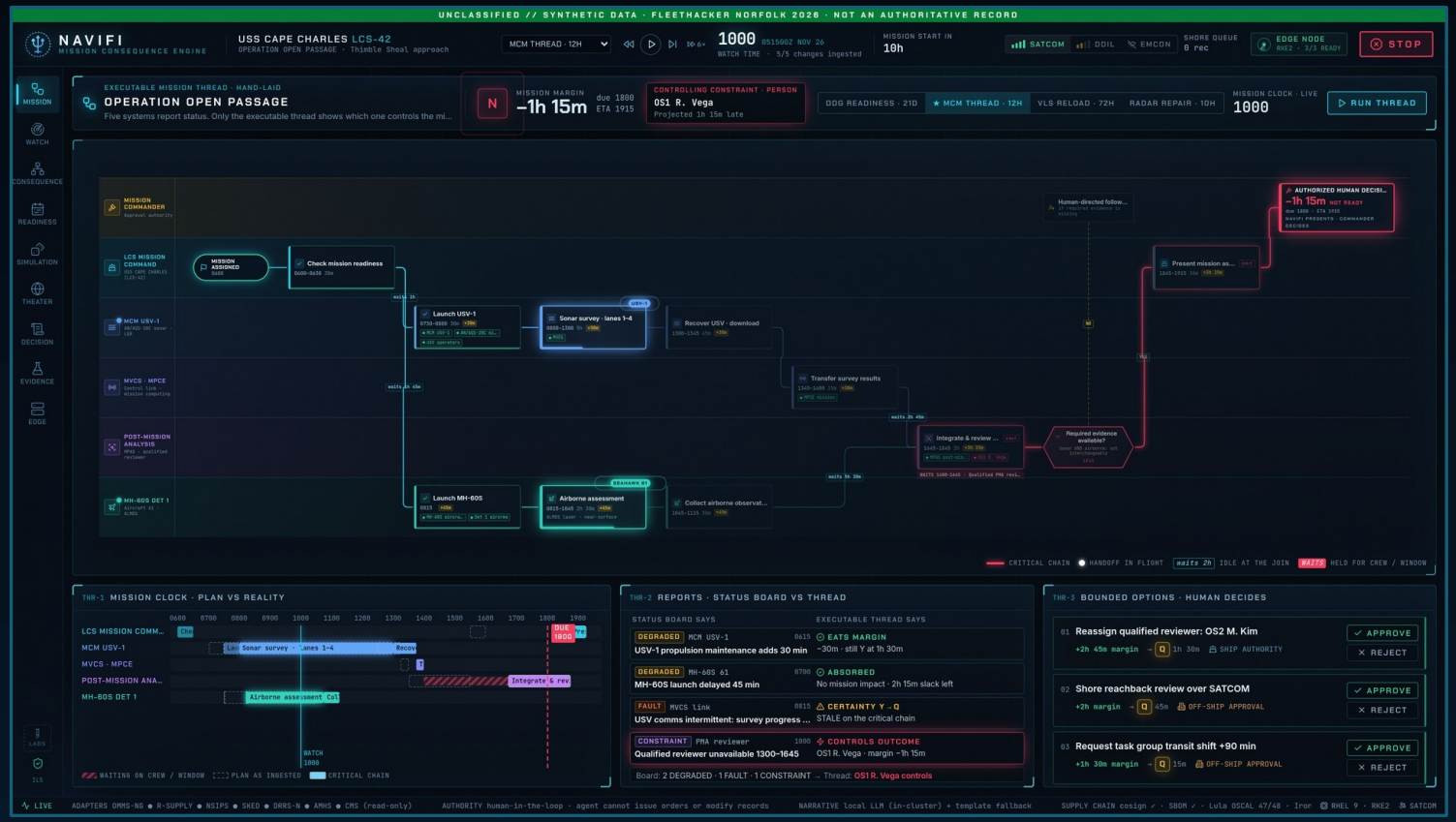
DRRS-N · WATCHBILL · MVCS · AMHS

Eight systems in one picture. Synthetic records for now.

WHAT WE BUILT

We made the mission itself executable

Operation Open Passage: mine countermeasures for USS CAPE CHARLES (LCS-42), synthetic data



The live app at 1000, mid-mission

1 Six swimlanes, one thread

Mission commander to the MH-60S detachment, recomputed on every status report.

2 Four reports in. One controls.

At 1000 it points at the analyst's review window, not the degraded USV or helo: −1h 15m, not ready.

3 Fixes simulated, not guessed

Three bounded options run on the same thread. Reassigning a qualified reviewer gets to +1h 30m.

WHO DECIDES

The AI explains. A person decides.

We kept the AI out of the math on purpose.

Deterministic engine

Same inputs, same answer, every time.

The AI only narrates

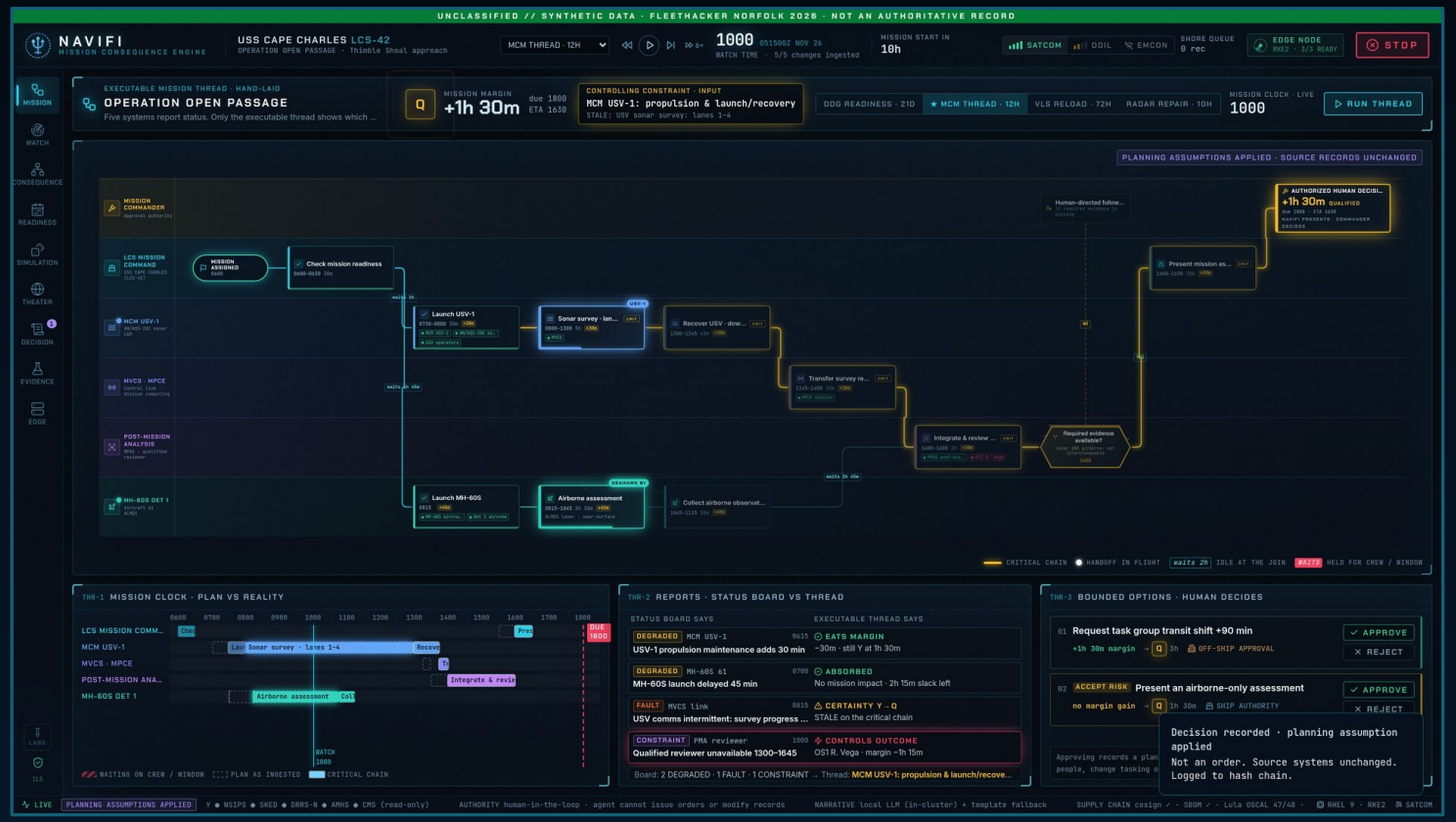
Switch the language model off and the answer is identical.

Approve, reject or STOP

Approving applies a planning assumption, never an order. Source records stay untouched.

Every decision on a hash chain

Tamper with the log and verification catches it, then restores from the edge copy.



After a person approved one option: −1h 15m became +1h 30m, and the decision went on the log

THEN WE TRIED TO BREAK IT

Twelve ways a real watch could trip it up

We wrote the failure cases down, and the app runs every one of them live against the engine.

Bad data

- Missing input
- Stale input
- Contradictory sources
- Malformed record

Bad conditions

- Disconnected (EMCON)
- LLM unavailable

Hard problems

- Nonlinear consequence
- Hidden cliff
- No single fix works

Known answers

- Classic baseline
- Part slips to 1330
- Team slips to 1600



EVD-2 FAILURE CASES · RUN LIVE AGAINST THE ENGINE — 12/12 PASS — ▷ RUN ALL

CASE	INJECTED	EXPECTED	OBSERVED (LIVE)	RESULT	MS
Classic: baseline	Part 1200 · team 1400 · repair+test 3h · required 1800	Ready 1700, +1h margin, team controls	Ready 1700 · +1h · Qualified repair team available	✓ PASS	0.2
Classic: part slips to 1330	Part ETA 1200 → 1330	No mission consequence (upstream slack absorbs it)	Ready 1700 · margin still +1h	✓ PASS	0.1
Classic: team slips to 1600	Team availability 1400 → 1600	Misses 1800 by 1h, NOT READY	Ready 1900 · -1h · N	✓ PASS	0.0
Missing input	Part ETA deleted from the feed	Flag MISSING, use conservative assumption, never silently READY	Flagged MISSING · assumed 1500 · Q at +0m	✓ PASS	0.0
Stale input	Team status last reported 0800; now 1100, staleness limit 2h	Flag STALE on the controlling input	Flagged STALE on repair team · status Q	✓ PASS	0.0
Contradictory sources	Watchbill says team 1400; AMHS message says 1500	Flag CONTRADICTORY, plan on the worse value	Flagged CONTRADICTORY · planned on 1500 → ready 1800	✓ PASS	0.0
Malformed record	Supply record arrives with date "2026-13-45T25:99"	Reject record, flag MALFORMED, keep last good value	Flagged MALFORMED · last good 1200 kept · no engine errors	✓ PASS	0.1
Disconnected (EMCON)	All network calls fail during the recompute	Full answer computed aboard, zero network calls	6 capabilities + options computed · 0 network calls	✓ PASS	5.2
LLM unavailable	Narrative model offline	Identical calculated answer; template explanation	Status, margin and control identical · LLM never in the calculation path	✓ PASS	0.4
Nonlinear consequence	CFA ETA slips 2d 3h (past the sail date)	Detect the missed Norfolk window; Rota controls; margin loss ≫ slip	2d 3h slip → -9d 18h margin · Rota port visit	✓ PASS	0.3
Hidden cliff	Helo detachment fly-on, currently on time	Warn that a small slip misses the fly-on window	Cliff after 2h slip · drop 5d 14h	✓ PASS	1.7

The failure-case table in the app, all twelve passing

We wrote down what it does not do yet, too

Durations are planner estimates, delays are treated as independent, and the picture is one ship. Those limits are on the evidence screen next to the results.

12 failure cases, run live

39 app tests

48 tests on Connor's engine

ONE ENGINE

Then we pointed it at three more missions

We laid out the MCM thread by hand. The same engine laid out the rest from their dependency graphs.

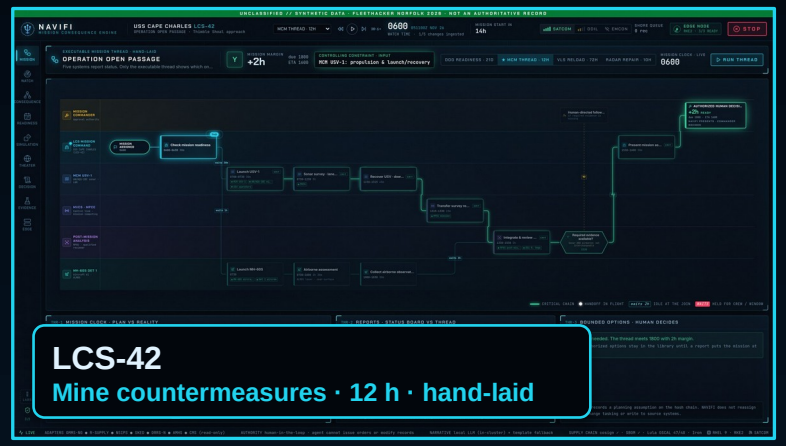
4 mission threads

2 ship classes

3 hulls

0 engine rewrites

Threads, interfaces and deployment profiles are configuration, so a new ship class is data work, not code.









NAVIFI · INFRASTRUCTURE

Built this week: a full Kubernetes platform on AWS

From terraform apply to a hardened cluster serving NAVIFI at **navifi.uds.demo.uncomn.ai**, all from code in the repo

1 TERRAFORM
Provision AWS

Node, firewall, IAM role, load balancer rules, TLS certificate and DNS

27 resources, one apply

→

2 FIRST BOOT
Harden the host

RHEL 9 with SELinux enforcing. SSM agent instead of SSH, no key pair

user-data.sh

→

3 RKE2
Install Kubernetes

Single-node RKE2 v1.36 with SELinux, plus the kernel setup Istio needs

install-rke2.sh

→

4 UDS CORE
Install the platform

Zarf init, then UDS Core 1.14: policy, mTLS, SSO, logs, metrics, backups

install-core.sh, signed

→

5 NAVIFI
Deploy the app

Any git commit: build, test, package with Zarf, roll out into the cluster

deploy.sh, SBOM per build

Where it runs

AWS us-east-1, inside the existing lab VPC

EC2 m7i.2xlarge: 8 vCPU, 32 GiB

RHEL 9 with SELinux enforcing

Behind an AWS load balancer with TLS

navifi.uds.demo.uncomn.ai

Real problems we solved on the way

SELinux locked a 2nd registry pod out of its volume
→ pinned the registry to one replica

RKE2 1.36's Gateway API CRDs blocked UDS Core
→ turned RKE2's bundled ingress off

MinIO stopped publishing public images
→ moved to the Chainguard build

Tight storage rules cut Loki off from the API
→ added explicit Kubernetes API rules

All in code

27 AWS resources from one Terraform apply

4 scripts from first boot to live app

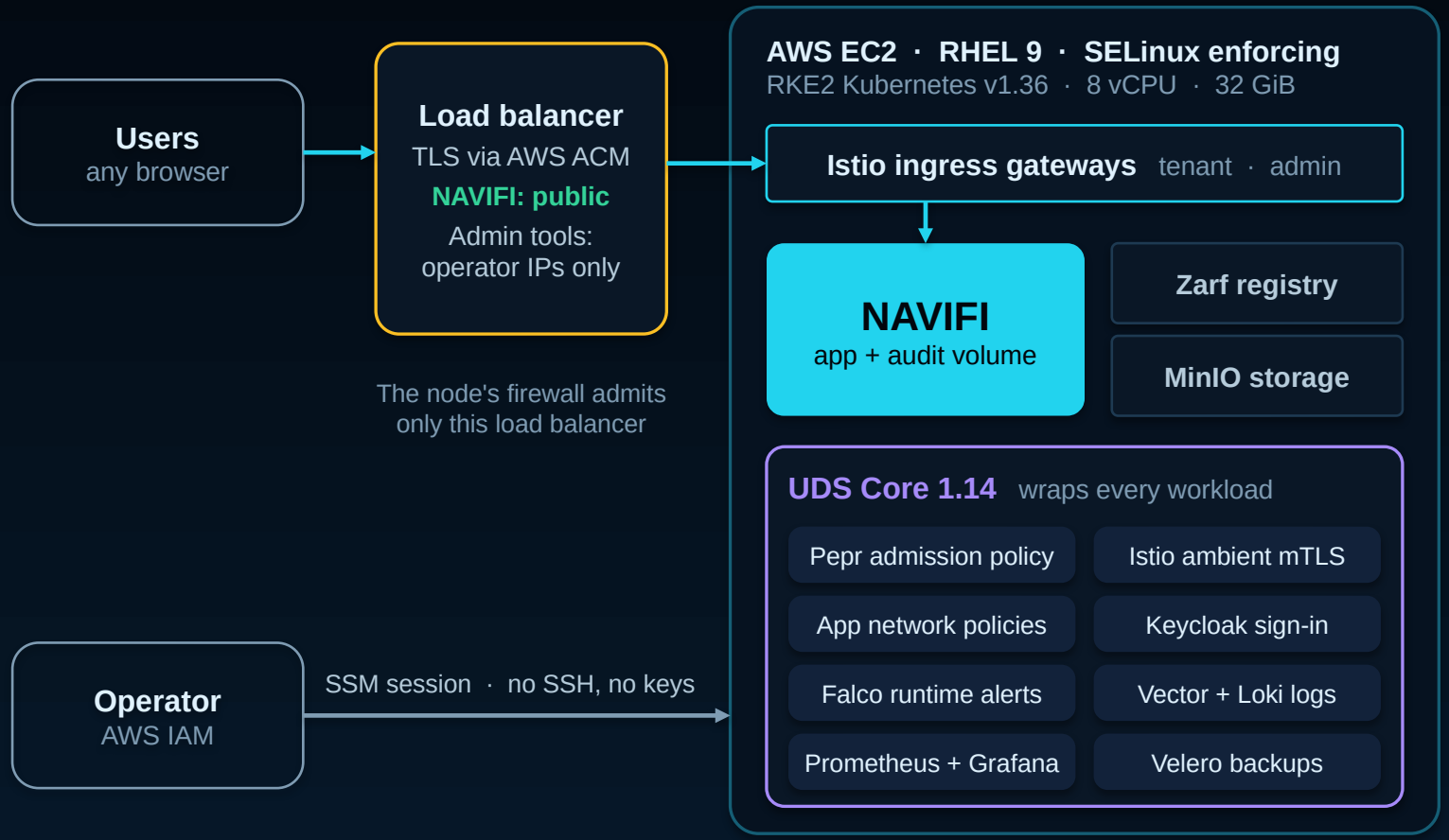
3 Zarf and UDS packages, platform to app

Rebuild it from infra/README.md

NAVIFI · LAB CLUSTER ON AWS

One hardened node running the ship's stack

RHEL 9 · RKE2 Kubernetes · Defense Unicorns UDS Core · Zarf live at **navifi.uds.demo.uncomn.ai**



Locked down by default

- SSM sessions only, logged in CloudTrail
- Only NAVIFI is public; admin tools return 404
- Default-deny network policy and mTLS

Commit to live in one command

- Typecheck and tests gate every image build
- Zarf packages it with an SBOM, air-gap style
- Every commit gets its own image tag

Mirrors the shipboard deploy

- Same RHEL 9, RKE2, UDS Core and Zarf
- Lab only: TLS at the load balancer, sign-in off
- Ship adds DISA STIG, FIPS and Keycloak SSO

8 vCPU · 32 GiB
the whole platform on one node

0
node ports open to the internet

Signed
Sigstore-verified, SBOM per build

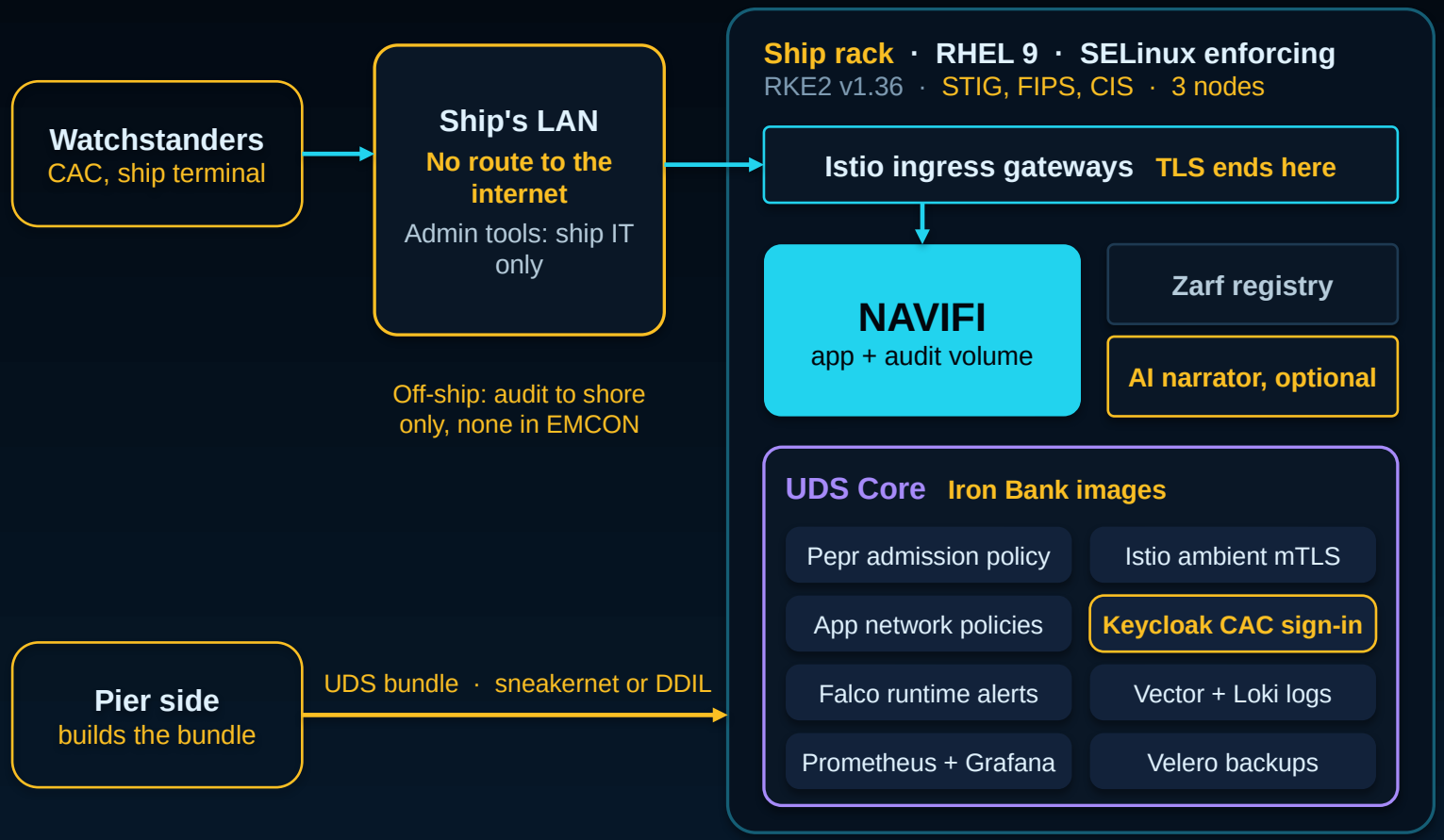
1 command
from any git commit to live

NAVIFI · THEORETICAL SHIPBOARD DEPLOYMENT

The same stack aboard, air-gapped

RHEL 9 · RKE2 Kubernetes · Defense Unicorns UDS Core · Zarf same as the lab

Amber: changes from the lab



Cut off by design

- No internet route; nothing downloads at sea
- CAC sign-in through Keycloak
- Default-deny network policy and mTLS

Works with the link down

- SATCOM: audit goes ashore in batches
- DDIL: one entry at a time, local model
- EMCON: nothing leaves, everything works

What changes from the lab

- One air-gapped bundle, not a git build
- TLS at the Istio gateway, no load balancer
- DISA STIG, FIPS, CIS and Iron Bank images

1 bundle
Zarf init, UDS Core and NAVIFI

0
downloads at sea, model included

Signed
Sigstore-verified, SBOM per build

3 link states
SATCOM, DDIL and EMCON

WHERE WE ARE

What's real, what isn't yet, and what's next

Three days in. Here is the straight version.

Real today

- Runs on the ship's stack, live on AWS
- Four mission threads on one engine
- Twelve failure cases and a hash-chained decision log
- Any git commit to live in one command

Not real yet

- All data is synthetic
- Durations are planner estimates, not fleet history
- Correlated delays (one cause, many slips) aren't modeled
- One ship at a time; squadron roll-up is a prototype

Next

- Wire Connor's new mission engine into the app
- 30 days: read-only exports from one DDG; three watch teams try it against today's process
- 60 days: workflow authoring, and UDS on an RKE2 edge node (we did a version of this one this week)
- 90 days: three hulls plus a squadron roll-up

Synthetic data only. A human decides.

APPENDIX

More detail, if you want it

Each one goes a level deeper on something from the main deck.

A1

Nothing typed twice

Where the data comes from, and what happens when it is bad

A2

What it gives back to the watch

Our before and after for one change landing mid-watch

A3

Built for the person on watch

The screen, the words, the STOP button, and EMCON

A4

Slack is not margin

The one idea under the engine

A5

Four things broke on the way to AWS

What we hit standing up the cluster, and the fixes

A6

Beyond the surface fleet

Connor's engine: other ships, other domains

APPENDIX A1 · WHERE THE DATA COMES FROM

Nothing typed twice

NAVIFI reads the records the ship already keeps. Nobody re-enters a status to make the picture move.

Read-only connectors

- **OMMS-NG** maintenance
- **R-SUPPLY** parts and ETAs
- **NSIPS** personnel
- **WATCHBILL** watches and quals
- **SKED** PMS schedule
- **DRRS-N** readiness
- **MVCS** unmanned vehicles
- **AMHS** message traffic

→ **Change event**
Time-stamped, with its source record attached

→ **Mission recomputed**
in milliseconds, every number linked to its record

Connectors only read. NAVIFI never writes back to a system of record.

Bad data gets flagged, not retyped

Missing	plans on a conservative value and says so
Stale	flagged on the input that controls
Contradictory	plans on the worse value, shows both
Malformed	rejected; the last good value is kept

EVD-5 PROVENANCE BY SOURCE SYSTEM

SKED	PMS schedule	9 rec · 0 ev	READ-ONLY
DRRS-N	Readiness reporting	3 rec · 0 ev	READ-ONLY
WATCHBILL	Watch / quals	3 rec · 1 ev	READ-ONLY
AMHS	Message traffic	1 rec · 0 ev	READ-ONLY
OMMS-NG	Maintenance (JCN / CASREP)	1 rec · 1 ev	READ-ONLY
MVCS	Unmanned vehicle control & data link	1 rec · 1 ev	READ-ONLY
HSC-DET	MH-60S detachment	1 rec · 1 ev	READ-ONLY
NSIPS	Orders, TAD, personnel	1 rec · 0 ev	READ-ONLY
NAVIFI	Derived by NAVIFI	1 rec · 1 ev	READ-ONLY

All records are synthetic and shaped like their source systems. Connectors read; nothing writes back.

The provenance panel: how many records each system supplied, all read-only.

The paperwork comes from the code

- SBOM and signature on every image build
- 48 NIST 800-53 controls in an OSCAL file generated from the code
- CI fails if that file drifts from the code

APPENDIX A2 · BEFORE AND AFTER

What it gives back to the watch

Our tabletop comparison for one change landing mid-watch. Not measured fleet data yet.

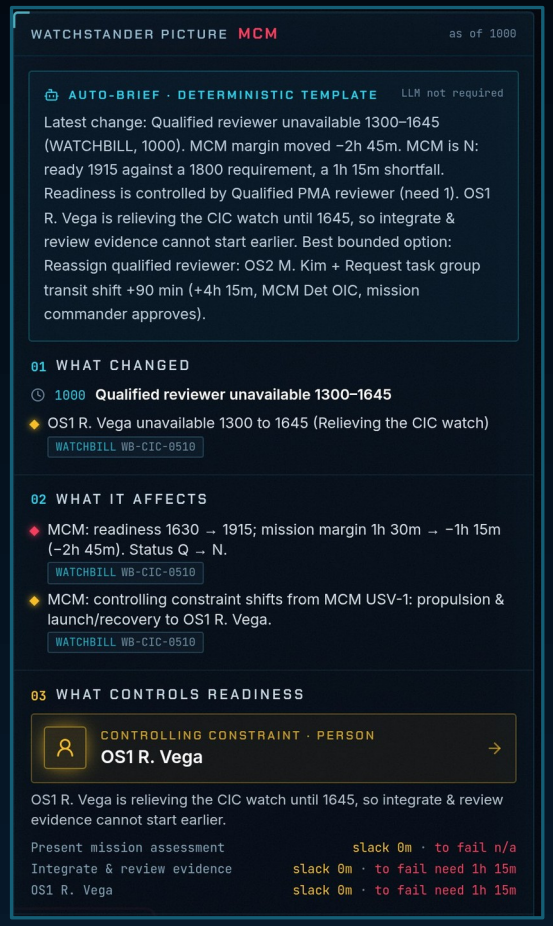
AFTER A CHANGE LANDS	TODAY	WITH NAVIFI	HOW
Time to a defensible answer	30–60 min	**Milliseconds**	Recomputed the moment the report lands
Systems consulted	5–7, plus calls	**One screen**	Every number links to its source record
Nonlinear effects caught	Often missed	**2 of 2**	A missed sail window and a helo fly-on cliff
Data problems surfaced	When they bite	**4 of 4**	Missing, stale, contradictory, malformed
Decision record	Notebook or email	**Hash-chained log**	Who decided what, on which facts, and when

How we would make it real In the first 30 days: sanitized, read-only exports from one DDG, and three watch teams run this same comparison against today's process.

APPENDIX A3 · THE SAILOR AT THE CONSOLE

Built for the person on watch

One screen, plain words, and it keeps working when the ship goes quiet.



The written brief on the watch floor

1 Five answers, always in this order

Is it up now? Will it be up when needed? What controls it? How much margin? What are the options?

2 Plain words first

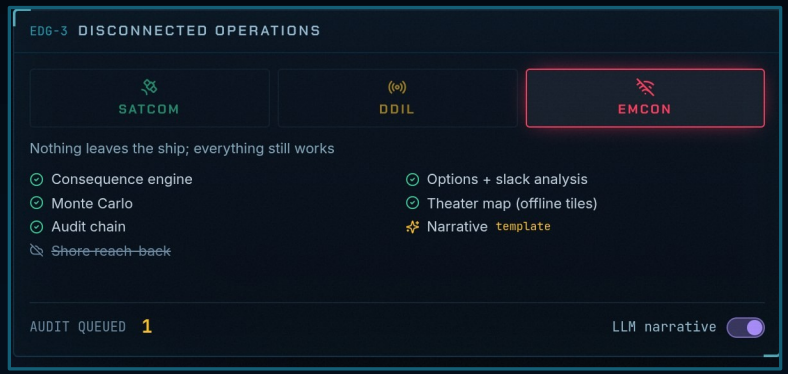
A written brief on top, numbers underneath. AI text is violet and labelled; with no model it falls back to a template.

3 One button to stop it

STOP halts every recommendation at once, and the stop itself is logged.

4 Hands stay on the keyboard

1–9 jump between screens, Space plays the change stream, Shift+R resets the session.



Flip to EMCON: nothing leaves the ship, and everything still works.

What we haven't done yet

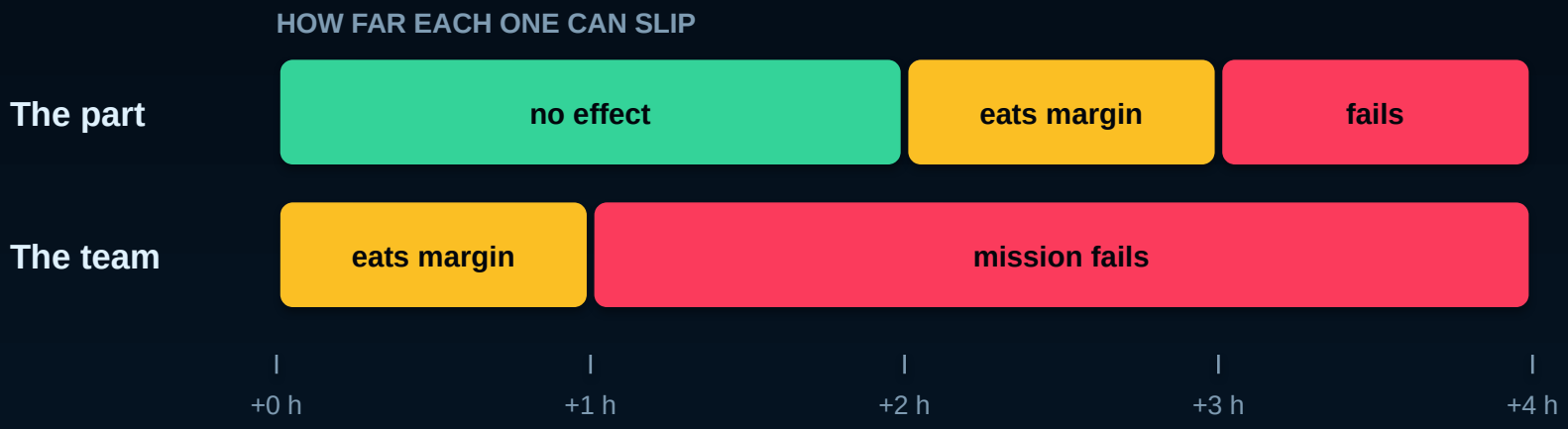
Put it in front of a real watch team. That is the first thing in the 30-day plan.

APPENDIX A4 · THE IDEA UNDER THE ENGINE

Slack is not margin

The late part gets the attention. The input that controls the mission often isn't the late one.

A radar repair: part due 1200, qualified team free at 1400, a 3-hour job, needed by 1800. **Ready at 1700 with 1 hour of margin. The team controls, not the part.**



Slack

How far an input slips before anything changes. The part: 2 hours. The team: none.

Margin

How far the mission can slip before it fails. Here 1 hour, and the team sets it.

Exact, not estimated

Readiness rules only move one way, so bisection finds both numbers exactly for every input.



The same check in the app: OS1 Vega on the mine countermeasures thread

APPENDIX A5 · WHAT WE HIT

Four things broke on the way to AWS

Each fix is in the scripts now, so the next node will not hit them.

WED 21:58

MinIO's public images were gone

WHY

MinIO stopped publishing the images the storage chart pulled.

FIX

Pull the Chainguard build that uds-k3d uses.

WED 22:07

The registry kept failing

WHY

A second registry pod shared the volume, and SELinux relabels it for the newest pod, locking the older one out.

FIX

Pin the Zarf registry to one replica.

WED 23:03

UDS Core wouldn't install

WHY

RKE2 1.36 ships Gateway API CRDs in its Traefik chart, and they blocked UDS Core's own.

FIX

Set ingress-controller to none, which skips both charts.

THU 01:11

Logs and backups lost the API

WHY

Narrowing Loki and Velero to MinIO also removed the rule that let them reach the Kubernetes API.

FIX

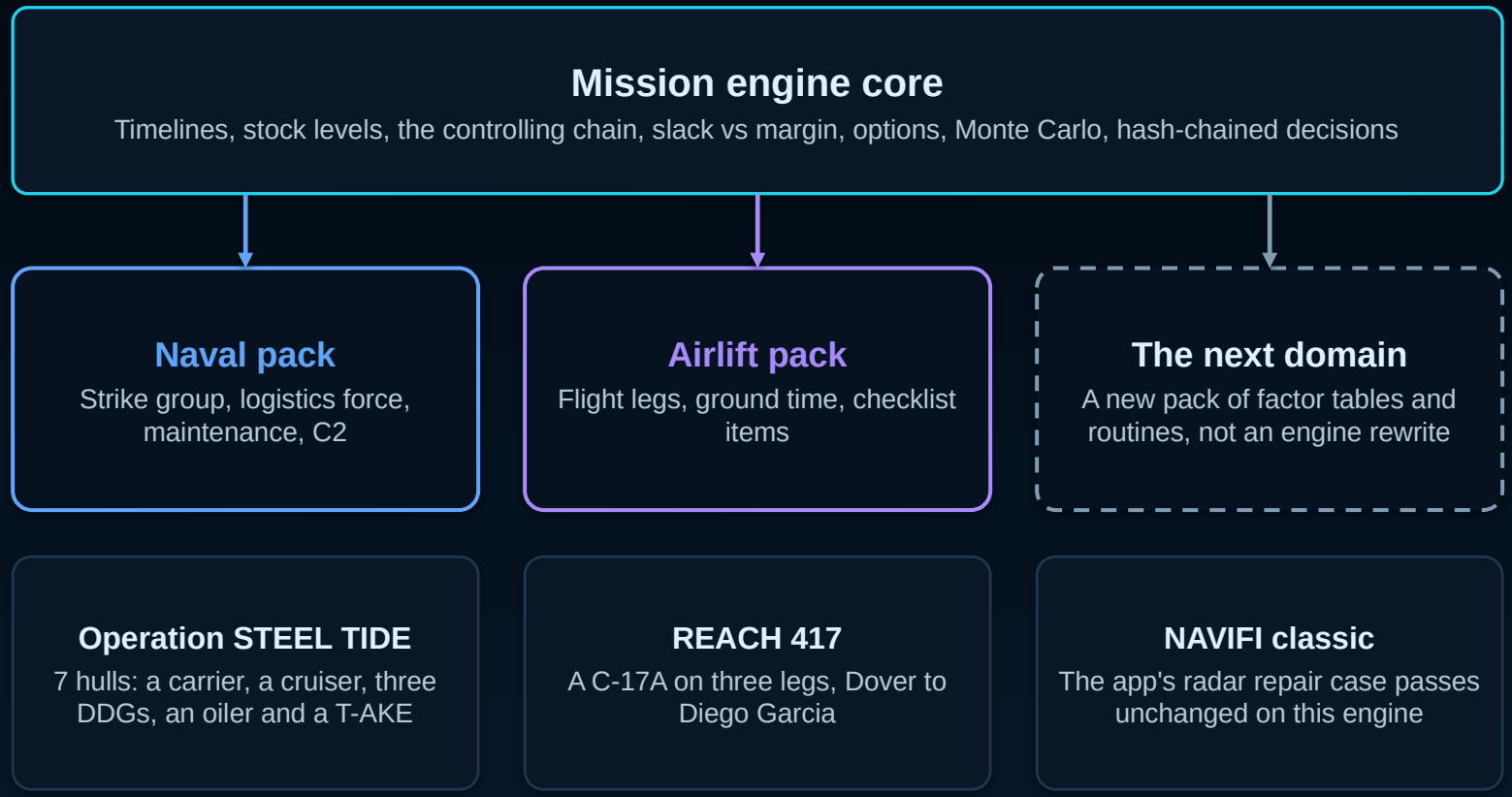
Add an explicit Kubernetes API rule for the pods that need it.

4 fixes	**10 of 10**	**45 pods**	**27**
each one now in the scripts	UDS packages ready	running and ready, none unhealthy	AWS resources from Terraform

APPENDIX A6 · OTHER SHIPS, OTHER DOMAINS

Beyond the surface fleet

Connor's engine keeps the core general and puts the domain knowledge in packs.



- Packs work together: the naval pack calls the airlift pack to check the carrier onboard delivery flight.
- Crews are tracked by qualification (AMD, strike, FC, ET), so who can cover a task is part of the math.

In the NAVIFI app today

- Four mission threads on one engine
- Two ship classes, three hulls
- A fleet picture prototype: 38 hulls, 12 missions, 140+ packages timed by the engine

Not yet

The app still runs its own engine. Wiring Connor's in is the next job; his engine's 48 tests run on their own today.